Data Acquisition
APIs & Web Scraping

Why Data Acquisition Matters

Most real-world data science projects don't start with clean CSV files.

Data acquisition is often the first and most time-consuming step in any data science project.

Modern data scientists need to know how to:

- Access data through APIs
- Scrape websites when APIs aren't available
- Handle authentication and rate limiting
- Build reliable data collection pipelines
- Work ethically and legally with web data

APIs

What is an API?

API = Application Programming Interface

An API is essentially a contract between you and a service:
"Here's how you can ask me for data, and here's how I'll respond"

Most modern APIs follow REST principles:
GET to retrieve data
POST to create
PUT to update
DELETE to remove

APIs return structured data (usually JSON)
Authentication via API keys keeps things secure
Rate limits prevent server abuse

Examples: GitHub, Twitter, Weather services, Financial data, Government databases

Why Use APIs?

Official, supported way to access data
Structured data format (usually JSON)
Real-time or near-real-time updates
Rate limiting prevents server overload
Authentication ensures authorized access
Documentation tells you exactly what's available
More reliable than web scraping
Legal and ethical - you're using the service as intended

Every API request has these components:
URL (the address you're sending to)
Method (GET, POST, PUT, DELETE)
Headers (authentication, format preferences)
Parameters (your specific query)
Response with status code (200=success, 404=not found, 429=rate limit)

REST API Basics

REST = Representational State Transfer

Most modern web APIs follow REST principles:

Resources are accessed via URLs (endpoints)
HTTP methods indicate the action:
GET: Retrieve data
POST: Create new data
PUT: Update existing data
DELETE: Remove data

Responses are typically in JSON format
Stateless - each request is independent

The requests library makes this beautifully simple in Python:

```
import requests
response = requests.get(url, params=params, headers=headers)
data = response.json()  # Automatically parses JSON to dict
```

HTTP Status Codes

Every API response includes a status code:

Success:
200 OK - Request succeeded
201 Created - New resource created

Client Errors:
400 Bad Request - Invalid request
401 Unauthorized - Authentication required
403 Forbidden - No permission
404 Not Found - Resource doesn't exist
429 Too Many Requests - Rate limit exceeded

Server Errors:
500 Internal Server Error - Server problem
503 Service Unavailable - Server down

API Endpoints

Endpoints are URLs that provide access to specific resources.

Example: GitHub API
Base URL: https://api.github.com

Common endpoints:
/users/{username} - Get user info
/repos/{owner}/{repo} - Get repository info
/repos/{owner}/{repo}/commits - Get commits
/search/repositories?q=python - Search repositories

Full URL example:
https://api.github.com/repos/python/cpython

Think of endpoints like a menu - each one gives you different data
Parameters let you customize what you get back

Simple API Request

```
import requests

# Public API - no authentication needed
url = 'https://api.github.com/repos/python/cpython'
response = requests.get(url)

if response.status_code == 200:
data = response.json()
print(f"Repository: {data['name']}")
print(f"Stars: {data['stargazers_count']}")
print(f"Forks: {data['forks_count']}")
else:
print(f"Error: {response.status_code}")
```

JSON Responses

APIs typically return data in JSON format.

JSON = JavaScript Object Notation
Human-readable text format
Easy to parse in any programming language
Supports nested structures

Example response:


```
{
"name": "cpython",
"stargazers_count": 45000,
"language": "Python",
"owner": {
"login": "python",
"type": "Organization"
}
}
```

Query Parameters

Query parameters customize API requests.

Added to URL after '?' and separated by '&'

Example:
https://api.github.com/search/repositories?q=python&sort=stars&order=desc

In Python with requests:

```
params = {
'q': 'python',
'sort': 'stars',
'order': 'desc'
}
response = requests.get(url, params=params)
```

The requests library handles URL encoding automatically.

Parameters let you filter, sort, paginate, and customize responses

API Authentication

Many APIs require authentication to:
Track usage and enforce rate limits
Provide access to private data
Prevent abuse

Common authentication methods:
API Keys - Simple token in header or URL
OAuth - More secure, user authorization
Bearer Tokens - Token in Authorization header

Never hardcode API keys in your code!
Use environment variables instead.

Example:

```
headers = {'Authorization': f'Bearer {os.getenv("API_KEY")}'}
response = requests.get(url, headers=headers)
```

Using API Keys

```
import requests
import os
from dotenv import load_dotenv

load_dotenv()  # Load from .env file

token = os.getenv('GITHUB_TOKEN')
headers = {'Authorization': f'token {token}'}

url = 'https://api.github.com/user/repos'
response = requests.get(url, headers=headers)

if response.status_code == 200:
repos = response.json()
for repo in repos[:5]:
print(f"- {repo['name']}")
```

Environment Variables

Store sensitive information in .env files:

```
# .env
GITHUB_TOKEN=ghp_your_token_here
TMDB_API_KEY=your_tmdb_key_here
OPENWEATHER_API_KEY=your_weather_key_here
```

Add .env to .gitignore:

```
# .gitignore
.env
```

Create .env.example as a template:

```
# .env.example
GITHUB_TOKEN=your_github_token_here
TMDB_API_KEY=your_tmdb_key_here
```

Rate Limiting

APIs limit how many requests you can make to prevent abuse.

Common limits:
GitHub: 60/hour (unauthenticated), 5000/hour (authenticated)
TMDB: 40 requests per 10 seconds
Twitter: Varies by endpoint

Check response headers for rate limit info:
X-RateLimit-Limit: Total allowed
X-RateLimit-Remaining: Requests left
X-RateLimit-Reset: When limit resets

Always implement delays between requests!

Simple approach:

```
import time
time.sleep(1)  # Wait 1 second between requests
```

Implementing Rate Limiting

```
import time

class RateLimitedAPI:
def __init__(self, calls_per_minute=10):
self.min_interval = 60.0 / calls_per_minute
self.last_call = 0

def get(self, url, **kwargs):
elapsed = time.time() - self.last_call
if elapsed < self.min_interval:
time.sleep(self.min_interval - elapsed)

self.last_call = time.time()
return requests.get(url, **kwargs)
```

Error Handling

Always handle potential errors:

```
try:
response = requests.get(url, timeout=10)
response.raise_for_status()  # Raises exception for 4xx/5xx
data = response.json()
return data
except requests.exceptions.Timeout:
print("Request timed out")
except requests.exceptions.HTTPError as e:
print(f"HTTP error: {e}")
except requests.exceptions.RequestException as e:
print(f"Request failed: {e}")
except ValueError:
print("Invalid JSON response")
```

Networks are unreliable - always use try/except
Always set a timeout - don't wait forever

Retry Logic with Exponential Backoff

```
def fetch_with_retry(url, max_retries=3):
for attempt in range(max_retries):
try:
response = requests.get(url, timeout=10)
response.raise_for_status()
return response.json()
except RequestException as e:
if attempt == max_retries - 1:
raise
wait_time = 2 ** attempt  # 1, 2, 4 seconds
print(f"Retry in {wait_time}s...")
time.sleep(wait_time)
return None
```

Handling Pagination

Many APIs return results in pages.

Example: GitHub returns 30 items per page

```
def get_all_repos(username):
all_repos = []
page = 1

while True:
url = f'https://api.github.com/users/{username}/repos'
params = {'page': page, 'per_page': 100}
response = requests.get(url, params=params)
repos = response.json()

if not repos:
break

all_repos.extend(repos)
page += 1

return all_repos
```

POST Requests

POST requests create new resources.

```
import requests
import json

url = 'https://jsonplaceholder.typicode.com/posts'

data = {
'title': 'My Data Science Project',
'body': 'Analysis of API usage patterns',
'userId': 1
}

response = requests.post(url, json=data)

if response.status_code == 201:
created = response.json()
print(f"Created post with ID: {created['id']}")
```

API Best Practices

Always check status codes before processing responses
Implement rate limiting and respect API limits
Use environment variables for API keys
Handle errors gracefully with try/except
Add timeouts to prevent hanging requests
Log all API calls for debugging
Cache responses when appropriate
Read the API documentation thoroughly
Test with small requests first

These aren't optional nice-to-haves - they're essential for production code

Start simple, add complexity only when needed
Make your code robust before making it fast

Free APIs for Practice

Great APIs for learning:

GitHub API - https://api.github.com
No key needed for basic use, 60 requests/hour

JSONPlaceholder - https://jsonplaceholder.typicode.com
Fake REST API for testing

REST Countries - https://restcountries.com
Country data, no authentication

OpenWeatherMap - https://openweathermap.org/api
Weather data, free tier available

TMDB - https://www.themoviedb.org/settings/api
Movie database, free API key

Web Scraping

What is Web Scraping?

Web scraping is extracting data from websites.

When to scrape:
No API is available
API doesn't provide the data you need
API is too expensive or restrictive

How it works:
1. Send HTTP request to get HTML
2. Parse HTML to find elements
3. Extract text, links, or other data
4. Save to structured format

Must be done ethically and legally!

Scraping is a tool, not a first choice
Always check for an API first - it's easier and more reliable

When to Scrape vs Use APIs

Prefer APIs when available:
More reliable and stable
Structured data format
Official and legal
Better performance

Scrape when necessary:
No API exists
API missing needed data
API too expensive
Public data on website

Never scrape:
Behind login walls (without permission)
Personal information
When Terms of Service prohibit it

Web scraping exists in a legal and ethical gray area
Being a good citizen of the internet isn't optional - it's essential

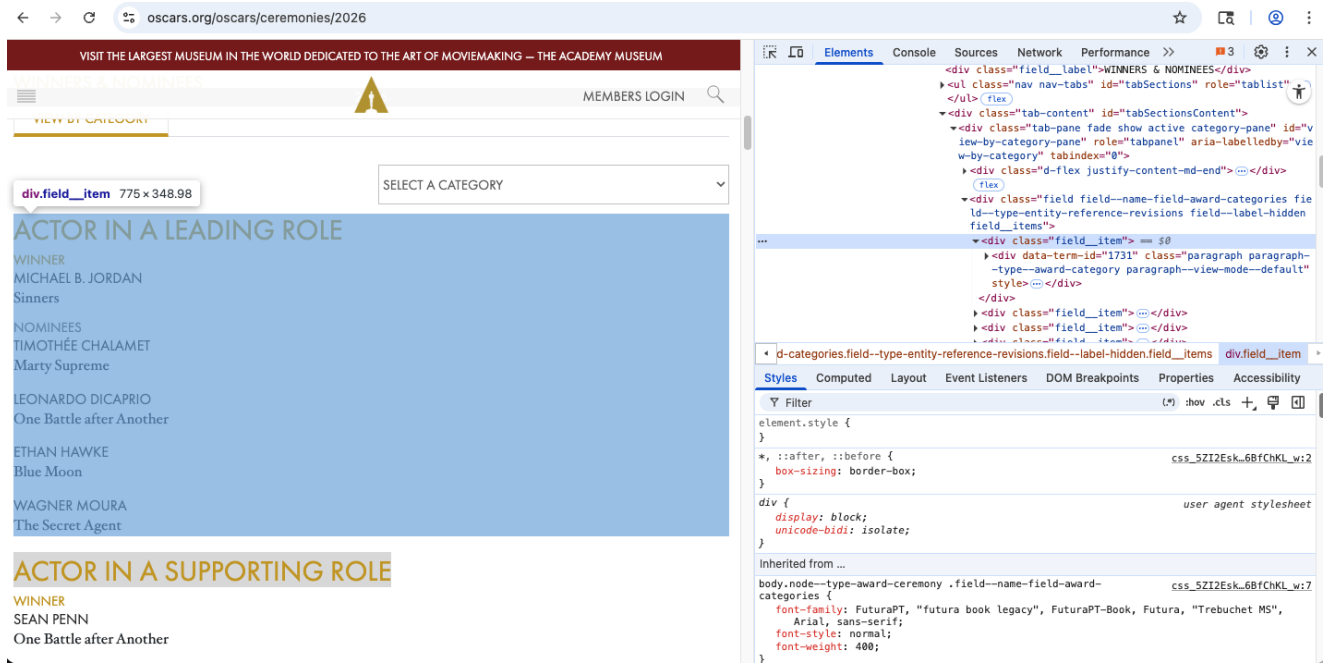
Using Chrome’s Inspect tool

The official list of nominees and winners of the 90th Academy Awards (in 2025) is available on the Academy of Motion Picture Arts and Sciences' website. By ocular inspection, this data is at least semi-structured: there are groups of nominees, each group has a title, each nominee has the name of the movie. But this is not structured in the way that we typically analyze data, a table with rows (observations) and columns (features).

This exercise assumes you're using Chrome, if you're using Safari (instructions) or Mozilla (instructions), there are slightly different workflows. If you right click on the "Actor in a Leading Role" and select "Inspect", a new window should appear on the side of your browser window. Make sure you have the "Elements" tab clicked.

Using Chrome's Inspect tool

You should see a nested list of collapsable elements. This is a very complex webpage, like many are these days, with many layers of tags. However if you mouse over the elements, things highlight in the primary browser window. You can see how this is really useful for matching specific HTML codes with what they produce. You can expand an element to see its sub-elements, sub-sub-elements, and so on.



Using Chrome’s Inspect tool

You should discover that the `<div class="view-grouping">` elements correspond to the nominee categories. The `"view-grouping"` is an attribute for the `div` tag. Expanding this element reveals a `<div class="view-grouping-content">` element, which then has seven "children": an `<h3>` (heading 3) tag for the name of the category, an obtuse `<div>` class for the winner, another `<h3>` tag for the nominees, and then four (or more) of the same obtuse `<div>` classes for the other nominees.

Expanding any one of the obtuse `<div>` elements reveals still more child tags that ultimately end with the name of the actor `<h4 class="field-content">Gary Oldman</h4>` and movie `<span class="field-content">Darkest Hour</span>`.

<The html for this site has been updated and is no longer represented by this format>

Forms of Structured Data

There are three primary forms of structured data you will encounter on the web: HTML, XML, and JSON.

HTML

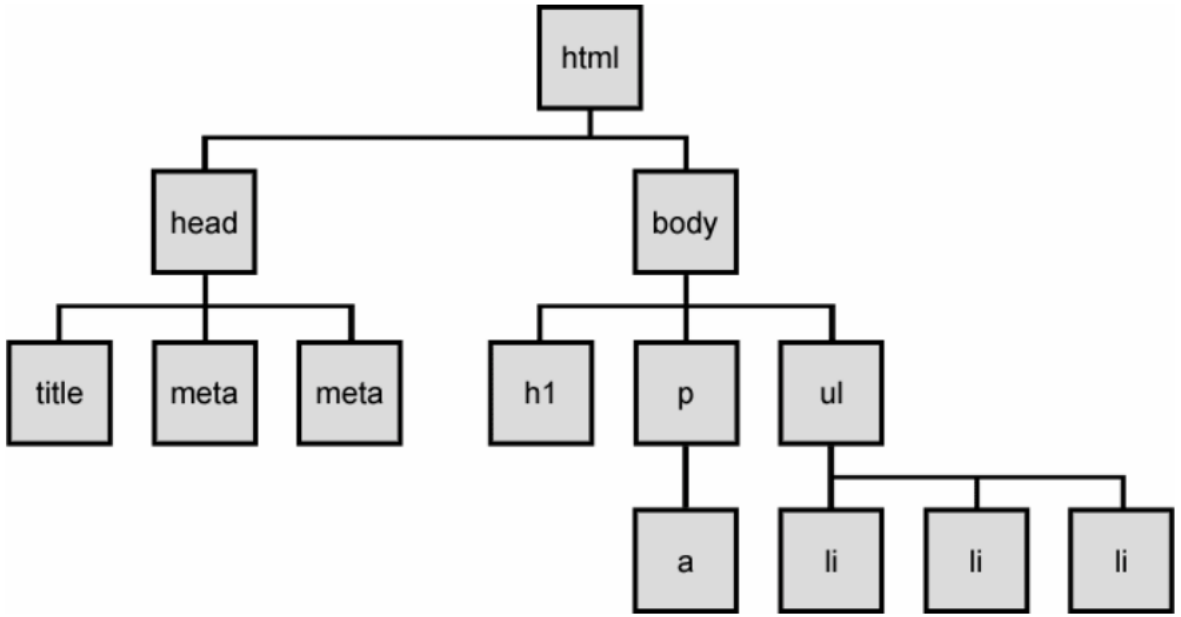
The Oscars example is an example of HyperText Markup Language (HTML), a markup standard that allows web browsers to render documents transmitted by a web server into webpages.

HTML is distinct from—but depends strongly on—the HyperText Transfer Protocol (HTTP) to move data from servers to clients.

HTML works with other standards like Cascading Style Sheets (CSS) and JavaScript to create the dynamic and interactive websites.

HTML

HTML has a tree-like structure with the "root" node being an `<html>` element with other elements like `<body>`s and `<head>`s as children, and those children having other children live the `<div>`, `<a>`, and other elements. Simple example of an HTML tree.



HTML Basics for Scraping

HTML = HyperText Markup Language

Structure:
<tag attribute="value">Content</tag>

Common tags:
 - Container
<p> - Paragraph
<h1>, <h2> - Headings
<a href="url"> - Links
<img src="url"> - Images
 - Inline container

Classes and IDs help identify elements:

<h2 class="title">Article Title</h2>

Use browser DevTools to inspect pages and find the right selectors

XML

The Extensible Markup Language (XML) is a related markup language for representing data structures.

XML is a robust and open—though verbose—standard for representing structured data.

The design goals of XML emphasize simplicity, generality, and usability across the Internet. It is a textual data format with strong support via Unicode for different human languages. Although the design of XML focuses on documents, the language is widely used for the representation of arbitrary data structures such as those used in web services. (https://en.wikipedia.org/wiki/XML)

JSON

JavaScript Object Notation (JSON) is probably the most popular data markup language and is especially ubiquitous when retreiving data from the application programming interfaces (APIs) of popular platforms like The New York Times, Reddit, Wikipedia, ***etc***.

HTML in the wild can be incredibly complicated and XML is verbose and has other idiosyncrasies

JSON is attractive for programmers using Python and R because it can represent a mix of different data types.

BeautifulSoup

Beautiful Soup is a Python library for parsing HTML.

Installation:

```
uv pip install beautifulsoup4 lxml
```

Basic usage:

```
from bs4 import BeautifulSoup
import requests

response = requests.get(url)
soup = BeautifulSoup(response.content, 'html.parser')

# Now you can search and extract data
title = soup.find('h1')
print(title.get_text())
```

Beautiful Soup makes HTML parsing almost fun
It's called Beautiful Soup because it makes sense of messy HTML soup

Finding Elements

Beautiful Soup provides multiple ways to find elements:

```
# Find first matching element
title = soup.find('h1')
article = soup.find('div', class_='article')
link = soup.find('a', id='main-link')

# Find all matching elements
paragraphs = soup.find_all('p')
links = soup.find_all('a')
articles = soup.find_all('div', class_='article')

# Get text content
text = title.get_text()
# Get attribute
href = link.get('href')
```

CSS Selectors

CSS selectors provide powerful element selection:

```
# Select by tag
soup.select('p')  # All <p> tags

# Select by class
soup.select('.article')  # class="article"

# Select by ID
soup.select('#main-content')  # id="main-content"

# Nested selection
soup.select('div.article > h2')  # Direct children
soup.select('div.article h2')  # All descendants

# Attribute selection
soup.select('a[href^="https"]')  # Links starting with https
```

The more specific your selector, the less likely it breaks when pages change

Simple Scraping Example

```
import requests
from bs4 import BeautifulSoup

# Set up headers to avoid being blocked
headers = {'User-Agent': 'Mozilla/5.0 (Windows NT 10.0; Win64; x64)'}

# Make request to 2026 Oscars page with headers
url = 'https://www.oscars.org/oscars/ceremonies/2026'
response = requests.get(url, headers=headers)

# Parse the HTML content
soup = BeautifulSoup(response.text, 'html.parser')

# Find title
title = soup.find('h1')
print(f"Title: {title.get_text()}")

#Get ceremony title
title = soup.find('h1')
print(f"Ceremony: {title.get_text()}")
```

Simple Scraping Example Cont

```
categories = soup.find_all('div', {'data-term-id': True})
print(f"\nTotal categories: {len(categories)}")

for cat in categories:
    cat_name = cat.find('div', class_='field--name-field-award-category-
   oscars')
    if cat_name:
        print(cat_name.get_text().strip())
```

Simple Scraping Example Cont

```
for cat in categories:
    cat_name = cat.find('div', class_='field--name-field-award-category-oscars')
    if cat_name:
        print(f"\n{cat_name.get_text().strip()}")
    winner_section = cat.find('div', class_='winner')
    if winner_section:
        parent = winner_section.find_parent('div', class_='paragraph--type--award-honoree')
        if parent:
            person = parent.find('div', class_='field--name-field-award-entities')
            person_name = person.find('div', class_='field__item').get_text().strip() if person and
    person.find('div', class_='field__item') else "N/A"
            film_wrapper = parent.find('div', class_='field--name-field-award-film')
            film_name = film_wrapper.get_text().strip() if film_wrapper else "N/A"
            print(f"  Winner: {person_name}")
            print(f"  Film: {film_name}")
```

ACTOR IN A LEADING ROLE

WINNER

MICHAEL B. JORDAN

Sinners

Extracting Structured Data [Pseudo Code]

```
def scrape_article(url):
response = requests.get(url)
soup = BeautifulSoup(response.content, 'html.parser')

article = {
'title': None,
'author': None,
'date': None,
'content': []
}

# Extract title
title_elem = soup.find('h1')
if title_elem:
article['title'] = title_elem.get_text()

return article
```

Handling Missing Elements [Pseudo Code]

Always check if elements exist before accessing:

```
# Bad - will crash if element missing
title = soup.find('h1').get_text()

# Good - safe handling
title_elem = soup.find('h1')
if title_elem:
title = title_elem.get_text()
else:
title = None

# Even better - with default
title_elem = soup.find('h1')
title = title_elem.get_text() if title_elem else 'No title'
```

Real websites are messy - data is often missing or malformed
Defensive coding prevents crashes

Cleaning Scraped Text [Pseudo Code]

```
import re

def clean_text(text):
if not text:
return ''

# Remove extra whitespace
text = ' '.join(text.split())

# Remove special characters if needed
text = re.sub(r'[^\w\s\-.,!?]', '', text)

return text.strip()

# Usage
raw_text = soup.find('p').get_text()
clean = clean_text(raw_text)
```

Handling Relative URLs [Pseudo Code]

```
from urllib.parse import urljoin

base_url = 'https://example.com'

# Relative URLs
relative_url = '/articles/123'
absolute_url = urljoin(base_url, relative_url)
# Result: https://example.com/articles/123

# Works with any relative path
link = soup.find('a').get('href')
full_url = urljoin(base_url, link)

# Handles edge cases
urljoin('https://example.com/page', '../other')
# Result: https://example.com/other
```

Pagination in Scraping [Pseudo Code]

```
def scrape_paginated_site(base_url, max_pages=5):
all_items = []

for page in range(1, max_pages + 1):
url = f"{base_url}?page={page}"
print(f"Scraping page {page}...")

response = requests.get(url)
soup = BeautifulSoup(response.content, 'html.parser')

items = soup.find_all('div', class_='item')
all_items.extend(items)

time.sleep(2)  # Be polite!

return all_items
```

Rate-Limited Scraper Class [Pseudo Code]

```
class RateLimitedScraper:
def __init__(self, delay=2.0):
self.delay = delay
self.last_request = 0
self.session = requests.Session()
self.session.headers.update({
'User-Agent': 'Educational Scraper'
})

def fetch(self, url):
elapsed = time.time() - self.last_request
if elapsed < self.delay:
time.sleep(self.delay - elapsed)

self.last_request = time.time()
return self.session.get(url)
```

User-Agent Headers

Always identify your scraper with a User-Agent header.

Bad (default):
User-Agent: python-requests/2.31.0

Good:

```
headers = {
'User-Agent': 'Mozilla/5.0 (Educational Scraper) '
'Contact: youremail@example.com'
}
response = requests.get(url, headers=headers)
```

Why it matters:
Identifies you to website owners
Shows you're not trying to hide
Provides contact info if there's an issue
Some sites block default user agents

robots.txt

robots.txt tells scrapers what's allowed.

Located at: https://example.com/robots.txt

Example:
User-agent: *
Disallow: /private/
Disallow: /admin/
Crawl-delay: 2

What it means:
Don't scrape /private/ or /admin/
Wait 2 seconds between requests

Always check robots.txt before scraping!
It's not legally binding but ethically important.

Checking robots.txt

```
from urllib.robotparser import RobotFileParser

def can_scrape(url):
rp = RobotFileParser()
rp.set_url(url + '/robots.txt')
rp.read()

return rp.can_fetch('*', url)

# Usage
if can_scrape('https://example.com/page'):
# Proceed with scraping
pass
else:
print("Scraping not allowed")
```

Robot.txt files?

Provides a sitemap to the robot to get other pages, it allows all kinds of User-agents, and disallows crawling of pages in specific directories (ads, polls, tests).

This is CNNs

```
print(requests.get('https://www.cnn.com/robots.txt').text)
```

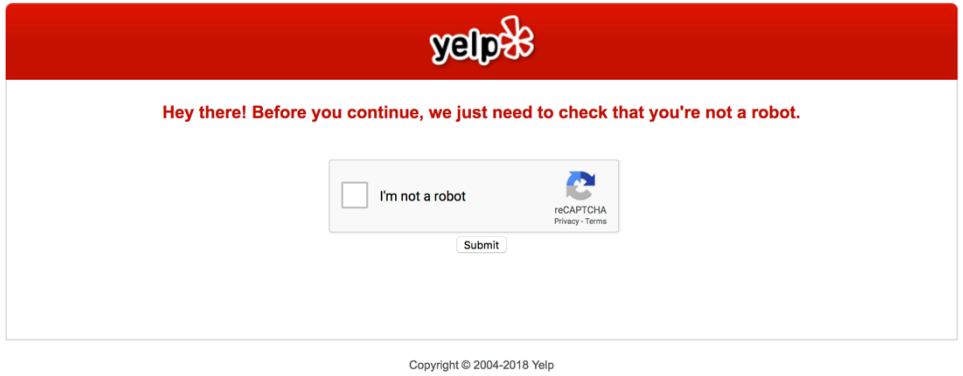
When scraping websites its worthwhile (and recommended to include a user-agent) so that the webmaster can get in contact

```
contact_header = {'User-Agent':'Python research tool by Nate
Langholz, langholz@g.ucla.edu’}

request =
requests.get('https://www.cnn.com',headers=contact_header)
```

Other things you should do:

- Reading the Terms of Service and Privacy Policies for the site's rules on scraping.
- Inspecting the robots.txt file for rules about what pages can be scraped, indexed, etc
- Be gentle on smaller websites by running during off-peak hours and spacing out requests
- Identify yourself by name and email in your User-Agent strings



Ethics of Web Scraping

The phrase "data scraping" is colloquial and popular but has pejorative connotations.

Data is valuable: other people invested time in collecting, organizing, and sharing it.

When you show up with a scraper you built after maybe a dozen hours demanding data, you rarely pay the costs of labor, hosting, *etc*. that went into making the data available.

There are *very* good rationales for making many kinds of data more available: reproducibility of scientific results, sharing publicly-funded and/or close-to-zero marginal cost resources, transparency and accountability in democratic institutions, remixing for innovative new analyses, *etc*.

Ethics of Web Scraping

But data breaches have become eponymous (Target in 2013, Equifax in 2017, Facebook in 2018, *etc*.) because they violate other values like privacy.

In the context of data scraping, there are really four ethical "areas of difficulty”:

- **Informed consent**: does the data scraper obtain consent from every person whose data is being retrieved?
- **Informational risk**: can the data scraper inflict economic, social, *etc*. harm on individuals by disclosing data?
- **Privacy**: does the data scraper know which information a person intended to be private or public?
- **Decision-making under uncertainty**: does the data scraper know all the ways the data could be (mis)used?

Ethical and Legal Risks

- **Copyright infringement**: compiling data that someone else can claim ownership over
- **Trespass**: over-aggressive scraping shuts down someone else's property
- **Computer Fraud & Abuse Act**: misrepresenting yourself to access a system is “hacking”

I cannot provide legal advice, we will revisit these concerns throughout the course through best practices for avoiding infringement, staggering data collection, simulating human requests, securing data, and protecting privacy

Ethical Web Scraping

James Densmore has a nice summary of practices for ethical web scraping:

- If you have a public API that provides the data I'm looking for, I'll use it and avoid scraping all together.
- I will always provide a User Agent string that makes my intentions clear and provides a way for you to contact me with questions or concerns.
- I will request data at a reasonable rate. I will strive to never be confused for a DDoS attack.
- I will only save the data I absolutely need from your page. If all I need it OpenGraph meta-data, that's all I'll keep.
- I will respect any content I do keep. I'll never pass it off as my own.
- I will look for ways to return value to you. Maybe I can drive some (real) traffic to your site or credit you in an article or post.
- I will respond in a timely fashion to your outreach and work with you towards a resolution.
- I will scrape for the purpose of creating new value from the data, not to duplicate it.

Ethical Scraping Guidelines

Check robots.txt before scraping
Implement rate limiting (minimum 1-2 seconds)
Use descriptive User-Agent headers
Respect Terms of Service
Only scrape public data
Handle errors gracefully
Don't overload servers with rapid requests
Don't scrape personal information
Don't ignore robots.txt directives
Don't use scraped data commercially without permission

Legal Considerations

Web scraping exists in a legal gray area.

Generally legal:
Scraping publicly available data
Following robots.txt
Not circumventing security measures
Using data for research/analysis

Potentially illegal:
Violating Terms of Service
Scraping copyrighted content
Accessing data behind authentication
Causing harm to the website

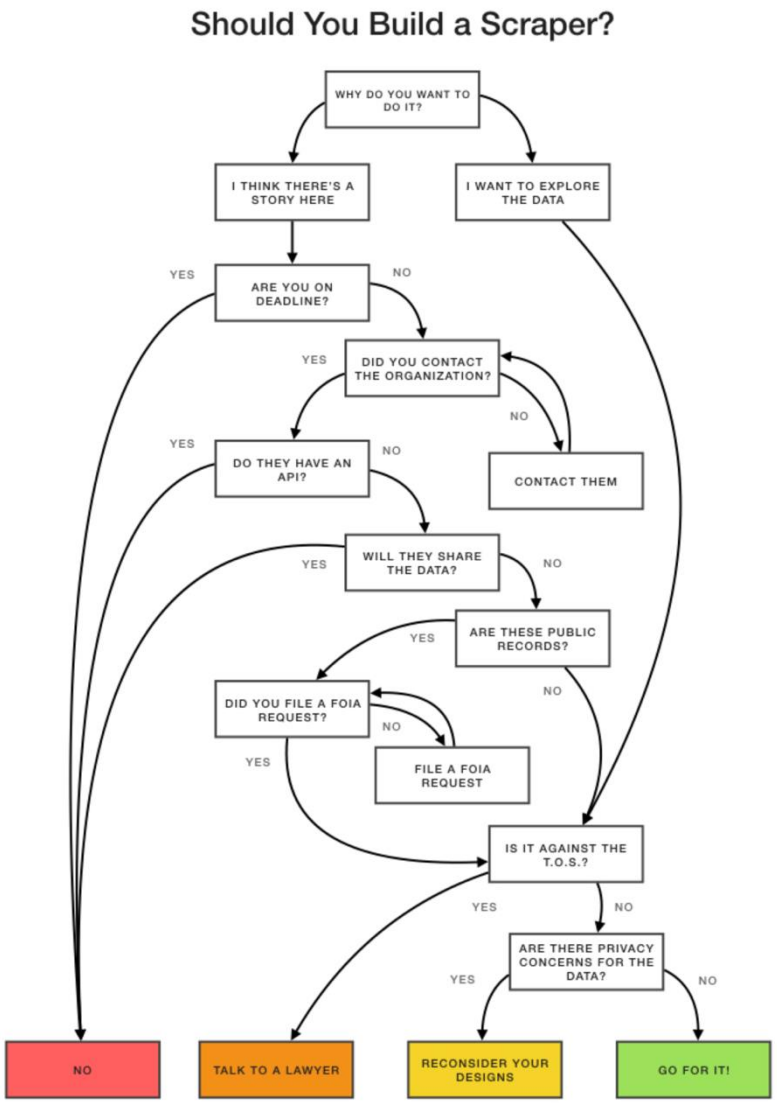
When in doubt, ask for permission or use an API!

Should you create a scaper?

Sophie Chou made this nice decision flow-chart of whether to build a scraper or not from a NICAR panel in 2016

There are some ethical justifications for violating a site's terms of service to scrape data:

- Obtaining data for the public interest from official statements, government reports, etc.
- Conducting audit studies (as long as these are responsibly designed and pre-cleared)
- The data is unavailable from APIs, FOIA requests, and other reports



Adverse consequences of web scraping include:

- Compromising the privacy and integrity of individual users' data
- Damaging a web server with too many requests
- Denying access to the web service to other authorized users
- Infringing on copyrighted material
- Damaging the business value of a web site

A history - Computer Fraud and Abuse Act

The Computer Fraud and Abuse Act was passed in 1984, in large part due to the 1983 film WarGames starring Matthew Broderick. A plain reading of the text of the law (18 U.S.C. § 1030) criminalizes just about any form of web scraping:

- Whoever intentionally accesses a computer without authorization or exceeds authorized access, and thereby obtains… information from any protected computer;
- knowingly causes the transmission of a program, information, code, or command, and as a result of such conduct, intentionally causes damage without authorization, to a protected computer;
- the term “exceeds authorized access” means to access a computer with authorization and to use such access to obtain or alter information in the computer that the accesser is not entitled so to obtain or alter;
- the term “damage” means any impairment to the integrity or availability of data, a program, a system, or information;
- the term “protected computer” means a computer which is used in or affecting interstate or foreign commerce or communication, including a computer located outside the United States that is used in a manner that affects interstate or foreign commerce or communication of the United States

Violators can be fined and jailed under a misdemeanor charge for up to 1 year for the first violation and jailed up to 10 years under a felony charge for repeated violations.

Computer Fraud and Abuse Act

This law has a chilling effect on many forms of research, journalism, and other forms of protected speech. The CFAA has been used by federal prosecutors to bring federal felony charges against programmers, journalists, and activists. In 2011, programmer and hacktivist Aaron Swartz (who contributed to the development of RSS, Markdown, Creative Commons, and Reddit) was arrested and charged with violating the CFAA for downloading several million PDFs from JSTOR over MIT's network. The decision to prosecute was unusual. Facing 35 years of imprisonment and over $1 million in fines under the CFAA, Swartz committed suicide on January 11, 2013.

Computer Fraud and Abuse Act

In 2016, four computer science researchers and the publisher of ***The Intercept*** who all use scraping techniques to run experiments to measure bias and discrimination in web content filed suit with the ACLU against the U.S. Government: ***Sandvig v. Sessions***. Their research involves creating multiple fake accounts, providing inaccurate information to websites, using automated tools to record publicly-available data, and other scraping techniques. In March 2018, the D.C. Circuit Court ruled two of the plantiffs had standing to sue and eventually ruled such scraping 'plausibly falls within the first amendment'

A federal court in another 2017 case, ***hiQ Labs v. LinkedIn,*** issued an injunction preventing LinkedIn from blocking data scraping and suggested the Computer Fraud and Abuse Act (CFAA) likely doesn't apply to scraping publicly available data. The ruling limits the use of the CFAA against scrapers but signals ongoing legal uncertainty in this area.

THE REAL JESSICA RYCHLY is a Minnesota teenager with a broad smile and wavy hair. She likes reading and the rapper Post Malone. When she goes on Facebook or Twitter, she sometimes muses about being bored or trades jokes with friends. Occasionally, like many teenagers, she posts a duck-face selfie.

But on Twitter, there is a version of Jessica that none of her friends or family would recognize. While the two Jessicas share a name, photograph and whimsical bio — "I have issues" — the other Jessica promoted accounts hawking Canadian real estate investments, cryptocurrency and a radio station in Ghana. The fake Jessica followed or retweeted accounts using Arabic and Indonesian, languages the real Jessica does not speak. While she was a 17-year-old high school senior, her fake counterpart frequently promoted graphic pornography, retweeting accounts called Squirtamania and Porno Dan.

All these accounts belong to customers of an obscure American company named Devumi that has collected millions of dollars in a shadowy global marketplace for social media fraud. Devumi sells Twitter followers and retweets to celebrities, businesses and anyone who wants to appear more popular or exert influence online. Drawing on an estimated stock of at least 3.5 million automated accounts, each sold many times over, the company has provided customers with more than 200 million Twitter followers, a New York Times investigation found.

The accounts that most resemble real people, like Ms. Rychly, reveal a kind of large-scale social identity theft. At least 55,000 of the accounts use the names, profile pictures, hometowns and other personal details of real Twitter users, including minors, according to a Times data analysis.

https://www.nytimes.com/interactive/2018/01/27/technology/social-media-bots.html

AI Copyright Lawsuits

COMMON CRAWL

The Data | Resources | Community | About | Search | Contact Us

Common Crawl maintains a free, open repository of web crawl data that can be used by anyone.

Common Crawl is a 501(c)(3) non–profit founded in 2007.

We make wholesale extraction, transformation and analysis of open web data accessible to researchers.

Overview

Dynamic Content Challenge

Some websites load content with JavaScript.

Problem:
requests.get() only gets initial HTML
JavaScript-loaded content won't be there

Solutions:
1. Find the underlying API
Check Network tab in browser DevTools
Look for JSON responses
Use that API directly

2. Use Selenium or Playwright
Automates a real browser
Executes JavaScript
Slower but more reliable

Finding Hidden APIs

Many dynamic sites use APIs behind the scenes.

How to find them:

1. Open browser DevTools (F12)
2. Go to Network tab
3. Filter by XHR or Fetch
4. Interact with the page
5. Look for JSON responses
6. Copy the request as cURL
7. Convert to Python requests

This is often easier than using Selenium!
You get structured JSON instead of parsing HTML.

Saving Scraped Data

```
import json
import csv

# Save as JSON
with open('data.json', 'w') as f:
json.dump(items, f, indent=2)

# Save as CSV
with open('data.csv', 'w', newline='') as f:
if items:
writer = csv.DictWriter(f, fieldnames=items[0].keys())
writer.writeheader()
writer.writerows(items)

# Save incrementally to avoid data loss
for item in items:
save_to_file(item)  # Save each item as you go
```

Logging Scraping Activity

```
import logging

logging.basicConfig(
filename='scraper.log',
level=logging.INFO,
format='%(asctime)s - %(levelname)s - %(message)s'
)

logging.info(f"Starting scrape of {url}")
logging.info(f"Found {len(items)} items")
logging.error(f"Failed to scrape {url}: {error}")
```

Benefits:
Track what you've scraped
Debug issues
Monitor progress
Identify patterns in failures

Common Scraping Issues

Issue: AttributeError: 'NoneType' object has no attribute 'get_text'
Solution: Check if element exists before accessing

Issue: Empty results
Solution: Verify selectors in browser, check for dynamic content

Issue: 403 Forbidden or 429 Too Many Requests
Solution: Add User-Agent, increase delays, respect rate limits

Issue: Inconsistent data
Solution: Add validation, handle missing fields gracefully

Issue: Encoding errors
Solution: Specify encoding: response.encoding = 'utf-8'

AI-Assisted Scraping

Use AI to speed up scraper development:

1. Provide HTML structure to AI
Copy HTML from browser inspector
Ask AI to write selectors

2. Describe desired output
"Extract title, author, date, and content"
AI generates parsing code

3. Debug with AI
Share error messages
Get suggestions for fixes

Always validate AI-generated code!
Test on small samples before full scrape.

Building Data Pipelines

A complete data acquisition pipeline:

1. Collection
API calls + web scraping
Error handling and retries
Rate limiting

2. Storage
Save raw data (JSON, CSV)
Timestamp everything
Keep original and processed versions

3. Processing
Clean and validate
Merge multiple sources
Handle missing values

4. Analysis
Load into pandas
Explore and visualize
Extract insights

Pipeline Example Structure

```
project/
 data/
   raw/          # Original scraped data
   processed/    # Cleaned data
   analysis/     # Results
 logs/
   scraper.log
 api_collector.py  # API data collection
 web_scraper.py    # Web scraping
 data_processor.py # Cleaning and merging
 analyze_data.py   # Analysis
 run_pipeline.py   # Orchestrate everything
 requirements.txt
 .env              # API keys (not in git!)
```

Combining APIs and Scraping

Often you need both:

Example: Movie data project
Use TMDB API for:
Movie metadata
Cast and crew
Official ratings

Scrape IMDb for:
User reviews
Additional ratings
Review sentiment

Merge on common identifier (IMDb ID)
Analyze correlations between sources

This gives you richer data than either source alone!

Data Validation

Always validate collected data:

```
def validate_movie_data(movie):
required_fields = ['title', 'year', 'rating']

# Check required fields exist
for field in required_fields:
if field not in movie or movie[field] is None:
return False

# Validate data types and ranges
if not isinstance(movie['year'], int):
return False
if not 1900 <= movie['year'] <= 2026:
return False
if not 0 <= movie['rating'] <= 10:
return False

return True
```

Handling Duplicates

```
import pandas as pd

# Load data
df = pd.read_json('movies.json')

# Check for duplicates
print(f"Total rows: {len(df)}")
print(f"Duplicates: {df.duplicated().sum()}")

# Remove duplicates
df = df.drop_duplicates(subset=['title', 'year'])

# Or keep first/last occurrence
df = df.drop_duplicates(subset=['id'], keep='first')

# Save cleaned data
df.to_csv('movies_clean.csv', index=False)
```

Caching API Responses

Avoid redundant API calls with caching:

```
import requests_cache

# Enable caching
requests_cache.install_cache(
'api_cache',
expire_after=3600  # 1 hour
)

# Now requests are cached automatically
response = requests.get(url)
```

Benefits:
Faster development and testing
Respect rate limits
Work offline
Reduce API costs

Testing Your Scraper

Always test on small samples first:

```
# Test with one URL
test_url = 'https://example.com/article/1'
result = scrape_article(test_url)
print(json.dumps(result, indent=2))

# Verify all fields are present
assert 'title' in result
assert 'author' in result
assert result['title'] is not None

# Test with a few URLs before full scrape
test_urls = urls[:5]
for url in test_urls:
result = scrape_article(url)
validate_result(result)
```

Monitoring Long-Running Scrapes

```
from tqdm import tqdm
import time

urls = get_urls_to_scrape()  # 1000 URLs
results = []

for url in tqdm(urls, desc="Scraping"):
try:
result = scrape_page(url)
results.append(result)

# Save incrementally
if len(results) % 100 == 0:
save_checkpoint(results)

time.sleep(2)
except Exception as e:
logging.error(f"Failed {url}: {e}")
continue
```